

OLTP vs OLAP Systems in Banking

A detailed, interview-ready guide with real banking use cases, architecture, examples, and SQL

1. Quick Understanding

OLTP (Online Transaction Processing) is designed to run the bank's day-to-day operational transactions such as deposits, withdrawals, fund transfers, card payments, loan payments, and account updates. The priority is fast, reliable, concurrent transaction processing with strong consistency.

OLAP (Online Analytical Processing) is designed to analyze large volumes of historical data. Banks use OLAP for management dashboards, customer analytics, fraud analysis, profitability, regulatory reporting, risk analysis, and business intelligence.

Aspect	OLTP	OLAP
Primary purpose	Run business transactions	Analyze business data
Banking example	Transfer ₹50,000 between accounts	Analyze monthly transfer trends
Data	Current operational data	Historical + integrated data
Workload	INSERT / UPDATE / DELETE / small SELECTs	Large SELECTs, joins, aggregations
Queries	Short and predictable	Complex and exploratory
Response	Usually milliseconds to seconds	Seconds to minutes, depending on workload
Concurrency	Very high	Usually lower than OLTP
Schema	Highly normalized	Often dimensional / denormalized
Optimization	Fast point lookups and writes	Fast scans, joins and aggregations
Transactions	ACID is critical	Read consistency and analytical correctness
Data volume	Usually operational subset	Very large historical datasets

2. What Happens in a Bank's OLTP System?

Imagine a customer transfers ₹10,000 from Account A to Account B. The banking application sends a transaction to the operational database. The system must validate the accounts, check available balance, debit Account A, credit Account B, record the transaction, and commit the complete operation. If one critical step fails, the transaction should not leave the database in an inconsistent state.

Typical OLTP tables

```
Customer(customer_id, name, mobile)
Account(account_id, customer_id, account_type, balance)
Transaction(transaction_id, from_account, to_account, amount, transaction_time, status)
Card(card_id, customer_id, card_type, status)
Loan(loan_id, customer_id, principal, interest_rate, outstanding_amount)
```

Why OLTP needs normalization

Operational systems are commonly normalized to reduce duplicate data and protect data integrity. For example, customer information should not be copied into every transaction row. Instead, a transaction stores customer/account identifiers and joins to the relevant master data when necessary.

3. What Happens in a Bank's OLAP System?

A bank usually extracts or streams data from many operational systems: core banking, cards, ATM systems, loan systems, mobile banking, internet banking, CRM, fraud systems, and payment gateways. That data is integrated into an analytical platform such as a data warehouse or lakehouse. Analysts and BI tools then run large queries over historical data.

Example analytical question

“What is the total transaction value by city, customer segment, product, and month for the last three years, and how has it changed compared with the previous year?”

This is not an ideal OLTP query because it can scan and aggregate millions or billions of transaction records. An OLAP platform is designed for this type of workload.

4. Banking Architecture: OLTP → ETL/ELT → OLAP

```
Source/OLTP systems → Core Banking / Cards / ATM / Loans / Payments
↓
Ingestion → CDC / APIs / Batch / Streaming
↓
Data Lake / Staging → Raw historical data
↓
Transformation → Cleansing / deduplication / business rules / enrichment
↓
OLAP / Warehouse / Lakehouse → Fact and dimension tables
↓
Consumption → Power BI / dashboards / analysts / risk / management / regulatory reporting
```

5. Important Banking OLTP Use Cases

Fund transfer: NEFT/RTGS/IMPS or internal transfers require accurate balance updates and transaction status management.

ATM withdrawal: The system checks account/card status and available balance, then records the withdrawal and updates the account.

Card payment: Authorization, balance/limit checks, merchant transaction recording, and payment status are operational workloads.

Loan repayment: Principal, interest, penalties, payment history, and outstanding balance are updated transactionally.

Deposit: Cash/cheque/online deposits update account balances and create transaction records.

Customer/account maintenance: Address, nominee, KYC status, account status, and product information are operational data changes.

6. Important Banking OLAP Use Cases

Customer 360: Combine deposits, loans, cards, payments, digital activity, and customer attributes to understand the customer.

Fraud analytics: Analyze transaction patterns, velocity, geography, merchant behavior, device information, and historical fraud signals.

Credit risk: Analyze repayment history, outstanding loans, income-related attributes, delinquency, and portfolio behavior.

Profitability analysis: Measure revenue, interest income, fees, costs, and profitability by customer, product, branch, or region.

Regulatory reporting: Prepare historical and aggregated information required for internal governance and regulatory processes.

Branch performance: Compare deposits, loans, transactions, customer acquisition, and revenue across branches and regions.

Product analytics: Determine which banking products are growing, which customer segments use them, and how product usage changes over time.

7. Banking Example: Transaction Analytics

OLTP table

```
transaction_id | account_id | transaction_type | amount | transaction_time | status
```

A typical OLTP query may retrieve one customer's latest transactions:

```
SELECT transaction_id, amount, transaction_time
FROM transactions
WHERE account_id = 101
ORDER BY transaction_time DESC
LIMIT 20;
```

OLAP query

An analytical query may calculate transaction value by month and transaction type:

```
SELECT
transaction_month,
transaction_type,
COUNT(*) AS transaction_count,
SUM(amount) AS total_amount
FROM fact_transactions
GROUP BY transaction_month, transaction_type
ORDER BY transaction_month;
```

The second query reads and aggregates much more data and is therefore much more appropriate for an OLAP environment.

8. OLAP Dimensional Model for Banking

A common warehouse design uses a **fact table** containing measurable events and **dimension tables** describing those events.

Example star schema

```
dim_customer(customer_key, customer_id, segment, city, age_group)
dim_account(account_key, account_id, account_type, branch_key)
dim_date(date_key, date, month, quarter, year)
dim_branch(branch_key, branch_name, city, region)
dim_transaction_type(transaction_type_key, transaction_type)

fact_transactions(transaction_key, customer_key, account_key, date_key, branch_key,
transaction_type_key, amount, fee)
```

This model makes it easier to answer questions such as “How much did savings-account customers transact in Madhya Pradesh during Q2?” without repeatedly joining many normalized operational tables.

9. Why Banks Usually Separate OLTP and OLAP

Running heavy analytical queries directly against the core banking database can compete with customer-facing transactions for CPU, memory, storage I/O, and database connections. A slow core banking database can affect ATM withdrawals, mobile banking, payments, and account operations. Therefore, organizations commonly move a copy or stream of operational data into a separate analytical environment.

10. Key Interview Differences

OLTP	OLAP
Operational / transactional	Analytical / decision support
Current state	Historical perspective

OLTP	OLAP
Many small transactions	Fewer but complex queries
High concurrent users	Analysts/BI/data workloads
Normalized schemas	Star/snowflake/denormalized models
Frequent writes	Mostly reads after data loading
Primary key indexes and point lookups	Columnar storage, partitioning, clustering, aggregations
Strict transaction processing	Large-scale analytical processing
Example: transfer money	Example: analyze transfer trends

11. How to Explain This in an Interview

Sample answer: “OLTP systems are used by banks to process real-time operational transactions such as fund transfers, ATM withdrawals, card payments, deposits, and loan repayments. They are optimized for high concurrency, low latency, data integrity, and ACID transactions. OLAP systems are used to analyze historical and integrated data for use cases such as fraud analytics, credit risk, profitability, customer 360, regulatory reporting, and management dashboards. In a typical banking architecture, data is captured from OLTP systems through CDC, APIs, batch, or streaming, landed in a lake or staging layer, transformed, and loaded into a warehouse or lakehouse. This separation prevents heavy analytical workloads from affecting customer-facing transactional systems.”

12. Simple Memory Trick

OLTP = Operate the Bank

Transfer money → withdraw cash → pay bill → update account → repay loan.

OLAP = Analyze the Bank

Which customers are profitable? → Which branches are growing? → What are fraud patterns? → How is loan risk changing? → What happened over the last 5 years?

13. Final Summary

The easiest way to distinguish the two is by the question the system answers. **OLTP answers: “What is happening right now, and can I process this transaction correctly?”** **OLAP answers: “What happened, why did it happen, and what patterns can we use for decisions?”** In banking, both are essential: OLTP keeps the financial operations running, while OLAP turns operational history into insights, controls, reports, and decisions.